

Parity and congruence periodicity of two integer recurrences obtained from matching counts in notched $3 \times n$ and $4 \times n$ grids: four theorems machine-checked in Lean 4 (claim of record)

Aurora0134*

September 28, 2026

Abstract

We record four theorems, machine-checked in Lean 4, about two recurrence-defined integer sequences $\mathbf{a3}$, $\mathbf{a4} \mathbb{N} \rightarrow \mathbb{Z}$, whose recurrences — order 5 and order 9 — were obtained by exact enumeration of the matchings of a $3 \times n$ and a $4 \times n$ grid graph with the top-right vertex deleted. Namely: $\mathbf{a3}(n)$ is odd exactly when $n \bmod 6 \in \{2, 3\}$; $\mathbf{a4}(n)$ is odd exactly when $n \bmod 5 \in \{0, 2\}$; $\mathbf{a3}(n + 12) \equiv \mathbf{a3}(n) \pmod{8}$; and $\mathbf{a4}(n) \equiv 2 \pmod{4}$ exactly when $n \bmod 10 = 6$. All four proofs compile with no placeholder and depend only on `{propext, Quot.sound, Classical.choice}`; the statements are byte-identical to a hash-frozen file. A structured novelty search found no occupying record for either sequence (adjudicated value tier: new sequence / new recurrence); the perfect-matching sub-problem of the $3 \times n$ notch is excluded from that because it is already OEIS A001353. The identification of the sequences with matching counts is *not* formalized. This note is a priority claim of record, not a full paper.

1 Introduction

Two graph families are considered: the grid graph $P_3 \square P_n$ and the grid graph $P_4 \square P_n$, each with its top-right vertex deleted (the “notch”). Their

*Contact: via the GitHub account Aurora0134; ORCID placeholder — to be filled in by the author before any upload. No personal information is carried in this note.

matchings (edge sets with no two edges sharing an endpoint, the empty set included) are counted two ways: a column-frontier dynamic program, which produced the whole 24-row count table, and a brute-force recursion over matchings (the least uncovered vertex left unmatched or paired with one of its neighbours), run over the leading eight and six entries respectively — there the two methods agree term by term. Berlekamp–Massey arithmetic over $\mathbb{Q}$ on those terms yields two integer linear recurrences, of order 5 for the $3 \times n$ family and order 9 for the $4 \times n$ family. Taking those recurrences and their initial values as *definitions*, this note records four theorems about the parities and residue classes of the two sequences, checked by the Lean 4 kernel.

The work was produced by AI4Math, a budgeted LLM-plus-kernel pipeline in which compilation is the sole adjudicator at every gate. Each theorem below carries a machine-readable anchor comment tying it to its Lean declaration name and to the auditor’s grade, *complete proof* for all four — quoted from the audit record, not a judgement made in this note.

This text is a priority claim of record: a short deposit note accompanying a Zenodo package, so that the two sequences and their recurrences are publicly timestamped while the slower paper lane proceeds (a fuller narrative paper on the same result, at `papers/notchgrid-parity-mod/`, will cite this deposit). Repository paths quoted in this note are internal ones: they are not publicly retrievable at deposit time, and the public mirror of this record is exactly the contents of this package. A literature survey, a prose proof and a research-level narrative are not duties of this note and are not attempted here.

2 Statement

Both sequences are functions $\mathbb{N} \rightarrow \mathbb{Z}$, not $\mathbb{N} \rightarrow \mathbb{N}$: the recurrences have negative coefficients, so the integer codomain lets the defining equations be read as ordinary subtraction on $\mathbb{Z}$, and it lets the `Int.ModEq` congruences and the `Odd` bridge of Section 3 apply without further coercion. Both are 0-indexed, so that `a3(0) = 2` and `a4(0) = 3` are the notch on one column. The two definitions, quoted verbatim from the artifact and character-identical to the frozen statement file, are:

Listing 1: `a3`: five initial values, order-5 recurrence (verbatim excerpt, lines 24–31 of `final/01-notch-proved.lean`).

1	<code>def a3 : ℕ → ℤ</code>
---	-----------------------------

```

2 | 0 => 2
3 | 1 => 10
4 | 2 => 67
5 | 3 => 407
6 | 4 => 2546
7 | n + 5 =>
8   5 * a3 (n + 4) + 9 * a3 (n + 3) - 9 * a3 (n + 2) - a3 (n +
   1) + a3 n

```

Listing 2: **a4**: nine initial values, order-9 recurrence (verbatim excerpt, lines 151–163). The order 9 is reported as found: it exceeds the order 3–6 the external proposal anticipated.

```

1 def a4 : ℕ → ℤ
2   | 0 => 3
3   | 1 => 32
4   | 2 => 407
5   | 3 => 4840
6   | 4 => 58608
7   | 5 => 705949
8   | 6 => 8515850
9   | 7 => 102684287
10  | 8 => 1238310540
11  | n + 9 =>
12    9 * a4 (n + 8) + 41 * a4 (n + 7) - 41 * a4 (n + 6) - 111 *
    a4 (n + 5)
13    + 91 * a4 (n + 4) + 29 * a4 (n + 3) - 23 * a4 (n + 2) - a4
    (n + 1) + a4 n

```

Both recurrences were fitted on 21 terms with 3 held out (residuals zero), refit on the minimal $2L$ -term window giving the same recurrence, and re-substituted against the independent dynamic program for all $n \leq 60$: computed evidence throughout. The theorems take the recurrences as definitions and are universal over $\mathbb{N}$.

Theorem 1 (parity of **a3**). *For every $n \in \mathbb{N}$, $\mathbf{a3}(n)$ is odd if and only if $n \bmod 6 = 2$ or $n \bmod 6 = 3$.*

Listing 3: Formal statement of Theorem 1: declaration header (verbatim excerpt, line 34).

```

1 theorem a3_odd_iff (n : ℕ) : Odd (a3 n) ↔ n % 6 = 2 ∨ n % 6 = 3
   := by

```

Theorem 2 (parity of $\mathbf{a4}$). *For every $n \in \mathbb{N}$, $\mathbf{a4}(n)$ is odd if and only if $n \bmod 5 = 0$ or $n \bmod 5 = 2$.*

Listing 4: Formal statement of Theorem 2: declaration header (verbatim excerpt, line 166).

```
1 theorem a4_odd_iff (n : ℕ) : Odd (a4 n) ↔ n % 5 = 0 ∨ n % 5 = 2
   := by
```

Theorem 3 ($\mathbf{a3}$ modulo 8 repeats with period 12). *For every $n \in \mathbb{N}$, $\mathbf{a3}(n+12) \equiv \mathbf{a3}(n) \pmod{8}$.*

Listing 5: Formal statement of Theorem 3: declaration header (verbatim excerpt, line 122).

```
1 theorem a3_mod8_periodic (n : ℕ) : a3 (n + 12) ≡ a3 n [ZMOD 8] :=
   by
```

Theorem 4 ($\mathbf{a4}$ is 2 modulo 4 in exactly one residue class). *For every $n \in \mathbb{N}$, $\mathbf{a4}(n) \equiv 2 \pmod{4}$ if and only if $n \bmod 10 = 6$.*

Listing 6: Formal statement of Theorem 4: declaration header (verbatim excerpt, line 281).

```
1 theorem a4_mod4_eq2_iff (n : ℕ) : a4 n ≡ 2 [ZMOD 4] ↔ n % 10 = 6
   := by
```

The residue tables over one full period starting at $n = 0$, computed from the definitions by exact integer iteration ($n \leq 400$), are

$$\begin{array}{ll} \mathbf{a3} \bmod 2 & (0, 0, 1, 1, 0, 0) & \mathbf{a3} \bmod 8 & (2, 2, 3, 7, 2, 6, 0, 0, 7, 7, 0, 0) \\ \mathbf{a4} \bmod 2 & (1, 0, 1, 0, 0) & \mathbf{a4} \bmod 4 & (3, 0, 3, 0, 0, 1, 2, 3, 0, 0) \end{array}$$

and they are also the pattern the Lean proofs work with: in Theorems 1, 2 and 4 the table appears as an inner lemma, whereas Theorem 3 is proved as a shift identity and contains no such table. Minimality of the periods 6, 5, 12, 10 is a computed fact and is *not* among the formalized statements; Theorem 3 asserts periodicity *with* period 12.

3 Proof

The complete development is `final/01-notch-proved.lean` (621 lines), shipped in full in this package at `proofs/01-notch-proved.lean`; the listing below is an *excerpt* of it. Three of the four proofs (Theorems 1, 2 and 4) share one residue-table skeleton: establish an inner statement (a **have** step, never an extra top-level theorem) that pins the residue of every term through an explicit **if-chain** over $n \bmod P$; enumerate the base window below the recurrence order with **interval_cases** and **decide**; in the inductive step write $n = j + L$ and unfold the recurrence exactly once by **simp only** [a3] or **simp only** [a4]; split on **mod_cases** $j \bmod P$, so that in each branch the table value of every predecessor is fixed and the weighted sum of the resulting congruences closes by one **decide**; then, in the two parity theorems, bridge back to **Odd** through **Int.odd_iff**. Listing 7 gives the proof of Theorem 3 complete, the most economical of the four, and it is *not* of that skeleton: no residue table, no **if-chain**, no **mod_cases** split, no **Int.odd_iff** bridge. It is a direct congruence calculation — one **simp only** [a3] step gives the recurrence equation, used at two offsets to expand both $a3(j + 12 + 12)$ and $a3(j + 12)$; the induction hypothesis applied to the five predecessors one by one; the five congruences combined through the compatibility of **Int.ModEq** with addition, subtraction and integer multiplication; closing by rewriting with the two expansions, the base window being $n < 12$ rather than the recurrence order. For Theorem 1 the artifact shows the residue-table skeleton at its most explicit (inner residue-table lemma with its induction setup at lines 35–45, one of the six residue branches at lines 51–60, bridge back to **Odd** at lines 106–119). Theorem 2 is Theorem 1 with nine predecessors and period 5 (its body is lines 167–277); Theorem 4 is the case where the table is genuinely four-valued, written as a three-level **if-chain** (inner lemma from line 282, ten forward branches to line 618, backward direction lines 619–621). Every numeric evaluation inside these proofs, the base-window values of Theorem 3 among them, is performed by the kernel’s own **decide** on closed integer expressions; there is no external oracle and no verification bound.

Listing 7: Proof of Theorem 3 in full (verbatim excerpt, lines 123–144; the complete source of all four proofs is in `proofs/01-notch-proved.lean`).

```
1 induction n using Nat.strong_induction_on with
2 | _ n ih =>
3   rcases lt_or_ge n 12 with hlt | hge
4   · interval_cases n <;> decide
```

```

5 · obtain ⟨j, rfl⟩ : ∃ j, n = j + 12 := ⟨n - 12, by omega⟩
6   have rec5 : ∀ m : ℕ, a3 (m + 5)
7     = 5 * a3 (m + 4) + 9 * a3 (m + 3) - 9 * a3 (m + 2) - a3
8     (m + 1) + a3 m := fun m => by
9       simp only [a3]
10      have e1 : a3 (j + 12 + 12)
11        = 5 * a3 (j + 23) + 9 * a3 (j + 22) - 9 * a3 (j + 21) -
12        a3 (j + 20) + a3 (j + 19) :=
13        rec5 (j + 19)
14      have e2 : a3 (j + 12)
15        = 5 * a3 (j + 11) + 9 * a3 (j + 10) - 9 * a3 (j + 9) -
16        a3 (j + 8) + a3 (j + 7) :=
17        rec5 (j + 7)
18      have h23 : a3 (j + 23) ≡ a3 (j + 11) [ZMOD 8] := ih (j +
19        11) (by omega)
20      have h22 : a3 (j + 22) ≡ a3 (j + 10) [ZMOD 8] := ih (j +
21        10) (by omega)
22      have h21 : a3 (j + 21) ≡ a3 (j + 9) [ZMOD 8] := ih (j + 9)
23      (by omega)
24      have h20 : a3 (j + 20) ≡ a3 (j + 8) [ZMOD 8] := ih (j + 8)
25      (by omega)
26      have h19 : a3 (j + 19) ≡ a3 (j + 7) [ZMOD 8] := ih (j + 7)
27      (by omega)
28      have key := (((h23.mul_left 5).add (h22.mul_left 9)).sub (
29        h21.mul_left 9)).sub h20).add h19
30      rw [e1, e2]
31      exact key

```

4 Verification evidence

- Toolchain: Lean v4.34.0 (leanprover/lean4:v4.34.0) [1], mathlib4 v4.34.0, revision 5ed29652 [2]; the pin is shipped verbatim as proofs/lean-toolchain.
- Statement integrity: the frozen statement file is formalized/01-notch-mods.lean (69 lines) and the delivered development is final/01-notch-proved.lean (621 lines); their sha256 values, which every listing in this note is checked against, are

```

9a787b6be05dc345343edf675d107801e674e3abae095356ace390fac8426fb8
6f1a7440f5830633e9309ab5629e16308928bb6421f64f4d4b1988b8929db4aa

```

The symmetric difference after stripping proof bodies is empty: the 6

top-level declarations (2 definitions and 4 theorem headers, names and order) are character-identical, and the delivered file adds no top-level statement.

- Kernel check: strict compile exits 0 with zero errors (320 deprecation warnings about `if_neg` and `if_pos`, recorded by the auditor as non-adjudicating); a byte-level text scan including comments finds no `sorry` and no `admit`; an escape-hatch scan (`set_option`, `axiom`, `unsafe`, `implemented_by`, `extern`, `macro_rules`, `elab`, `native_decide`, `decide!`) finds none. Axiom printout, copied verbatim and in the order the file prints it:

```
'a3_odd_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'a3_mod8_periodic' depends on axioms: [propext, Classical.choice, Quot.sound]
'a4_odd_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'a4_mod4_eq2_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
```

- Audit chain: gate 2 well-definedness (four gate-batch compiles accept the statements; $8 \times 4 = 32$ bare decision-procedure probes all fail, so no theorem is closable by one automation step; both sequences re-derived from the frozen definitions agree with the 24-row enumeration table in both columns and the four statements hold pointwise for $n = 0, \dots, 80$) and gate 3 final audit (symmetric difference, `sorry` and escape-hatch scans, re-run of the strict compile and of the axiom printout, both outputs byte-identical to the worker's own logs). What `audit/` of this package ships is the key-line extract of each gate, the verbatim axiom printout and the claim lane's own re-verification files; the gate 2 and gate 3 originals (`final-01-notch.txt`, `welldef-verdict.md`, `gate-t1.txt-gate-t4.txt`, `review-final-{strict,axioms}.out`) stay in the source task directory `tasks/20260927-mossad-notch/` and are not part of this deposit.

5 Novelty evidence

The pipeline's C9 novelty review adjudicated the two main propositions as *no occupying record found* (on its four-state scale: occupied / no occupying record found / known but not formalized / undecided), with value tier *new sequence* / *new recurrence* (a translated pool-card tier label; the Chinese original is in `audit/c9-record.md` of this package). This is a statement about search evidence, not a claim to new mathematics, and not a priority claim beyond the deposit itself. Per-channel summary, one line per

channel; the full log, including every zero-hit query, is copied verbatim into `audit/c9-record.md`:

channel	query	result
OEIS, sequence layer	a3 head 2, 10, 67, 407, 2546, 15782, 98104, 609304 and a4 head 3, 32, 407, 4840, 58608, 705949, each with four variants (drop 1, drop 2, $\times 2$, $+1$); divide-by-2 not applicable	all 10 queries zero-hit
OEIS, keyword layer	Hosoya index grid graph (3 hits), monomer dimer grid (10 hits)	no notched-count entry
OEIS, triangle check	A210662 rows and columns cut by hand: $T(3, \cdot) = 3, 22, 131$; $T(n, 3) = 131, 823, 5096, \dots$; $T(4, \cdot) = 5, 71, 823, 10012$ [4]	our heads are not segments
OEIS, derived layer	a3 near-perfect counts 4, 29, 161, 798 (even widths); a4 near-perfect 2, 7, 29, 88, 288 and its drop-1 tail	zero-hit
OEIS, derived layer (fence)	a3 <i>perfect</i> -matching counts, odd widths: 1, 4, 15, 56, 209, 780, 2911	A001353 occupied
literature, zbMATH	5 query families: monomer dimer grid (12), matching polynomial grid graph (90), matchings grid graph corner (1), deficient graph matchings (2), monomer dimer lattice vacancy (1)	no same proposition
literature, other	Math StackExchange number of matchings grid graph (6 items); arXiv monomer dimer grid graph (0)	none matching
literature, degraded	OpenAlex: HTTP 429 throughout (unverified, contributes no evidence either way); general web search: no search tool in that session, one engine DNS-poisoned, the other SEO noise	literature layer rests on zbMATH
library layer	mathlib (no GridGraph ; SimpleGraph/Matching.lean has matching predicates but no counting theorem), compfiles, sequencelib	zero hits
Lean ecosystem layer	GitHub repository search for Lean graph-matching formalizations	zero hits
channel self-check	0, 1, 1, 2, 3, 5, 8, 13 gives A000045; 1, 3, 13, 22, 44, 90, 196, 406 gives A180970	retrieval confirmed working

Two fences belong to the adjudication and are stated here, not in a footnote. (i) The *perfect*-matching sub-problem of the $3 \times n$ notch (odd widths) is OEIS **A001353**, $a(n) = 4a(n-1) - a(n-2)$ with $a(0) = 0$, $a(1) = 1$, whose record carries the independent attribution to Joshua Zucker and the Castilleja School Math Club (2003-10-28) for packing a $3 \times (2n-1)$

rectangle with one extra square attached [3]; nothing here claims that subproblem and no novelty statement extends to it. Our object is the *total* matching count. (ii) For the $4 \times n$ notch the vertex count $4n - 1$ is odd, so its perfect-matching count is identically zero: a degeneracy noted only to explain why that direction is empty, not a proposition.

6 Scope and limitations

1. **Grading boundary (quoted, not decided here).** The audit record grades all four theorems *complete proof*: no **sorry**, axioms inside the standard whitelist, statement byte-identical to the frozen text. That grade is the wording ceiling of this note.
2. **The semantic gap.** What is machine-checked is a property of two recurrence-defined integer sequences. That they *are* the matching counts of notched grids is computed evidence: the brute-force recursion agrees with the dynamic program term by term over the leading 8 entries of the $3 \times n$ column and the leading 6 of the $4 \times n$ column, which is the whole window it was run, so the remaining rows of the 24-row table are produced by the dynamic program alone; recurrence re-substitution was checked up to $n = 60$ and modular dynamic programming up to $n = 600$. The bridge is stated as a conjecture in the accompanying paper, not proved here. No sentence of this note should be read as a theorem of graph enumeration.
3. **Novelty is search evidence only**, at the adjudicated tier and with the two fences of Section 5; OpenAlex returned HTTP 429 and the general web layer was degraded, so if either later returns a strong hit the novelty sentence must be re-examined.
4. **The order-9 recurrence is reported as found.** It exceeds the order 3–6 the external proposal anticipated; the refit on the minimal 18-term window gives the same recurrence and it holds on the whole independent window, so it is not an artifact of a short fitting window: nothing was trimmed to make it look smaller.
5. **Computed versus checked.** Irreducibility of the two characteristic polynomials over $\mathbb{Q}$, minimality of the four periods, and every verification window are computed evidence (Python, exact integer or rational arithmetic), not Lean theorems.

6. **Provenance.** Proof search consumed no LLM sampling: the four proofs follow a skeleton already taken through the kernel on a sibling task of the same selection pool, and the auditor ruled that this lineage does not downgrade the grade. The autoformalization roundtrip check ran on the same model node as the formalizer, so it is not an independent review; the mitigation is that the kernel is the final arbiter and that the audit re-ran both compile and axiom printout.
7. This note contains no literature survey and no prose proof: it is a deposit record.

Data availability

Artifact package: Zenodo deposit (DOI to be reserved at upload; see `UPLOAD.md` in the source repository) containing this note (source and PDF), `proofs/` (frozen statement file, delivered development, toolchain pin), `audit/` (gate and compile evidence key lines, axiom printout, symmetric-difference conclusion, listing byte-verification reports, the claim audit verdict `claim-check.md`, the full C9 query record `c9-record.md`) and `metadata/`. A sanitized public mirror of exactly this package may be pushed to GitHub by the author, with the URL recorded in `claims/notchgrid-parity-mod/card.md` upon push; no other part of the pipeline is mirrored. The numerical layer that produced the two sequences (enumeration, fitting and modular probing scripts, 24-row count table) lives under `phase0/` of the source task directory, is not part of this deposit, and is likewise not publicly retrievable. Note under CC BY 4.0, code under MIT.

Use of generative AI

(a) *Stages done by AI:* candidate-proposition generation, the counting and fitting scripts, autoformalization into Lean statements, proof search, audit re-runs, prose translation and the drafting of this note were performed by LLM stages of the AI4Math pipeline; topic adjudication, statement-semantics verification and gate sign-off were human. (b) *Model, node and budget:* mathematical content was sampled on one resolved node, wire-id `anthropic/a6api-main/kimi-k3` (resolution level L3 static profile cross-checked against L2 relay state, taken 2026-09-27), with no cross-node switching. Numbers copied verbatim from the source ledger

`tasks/20260927-mossad-notch/budget.log`: llm-call 2/8 (both calls are autoformalization roundtrip re-translation checks, tokens in/out 1628/4000 truncated and 1628/1889 complete) and Lean compile 17/24 (gate 2: 6, proof search: 9, final audit re-runs: 2); Phase 0, proof search, audit and reporting used no sampling at all. This claim-note stage used llm-call 0 of its budget of 4 and 10 LaTeX compile rounds of its budget of 11 (the original packaging used the then cap of 6, which the user raised to 9 on 2026-09-28 for the rework done after the audit, and to 10 the same day with that tenth round reserved for the DOI backfill; on 2026-09-29 the user directed that the reserved tenth round be spent on the author-field errata instead and pre-authorized an eleventh round for the DOI backfill; the ninth round was spent on a package-promise wording correction, the tenth on the author-field errata; no cap was exceeded), as recorded in `claims/notchgrid-parity-mod/budget.log`. The drafting session ran on a different endpoint whose model selection key is `auto` (routing not pinned); it contributes no sampled claim, and every assertion here that something was “proved” or “passed” rests on kernel compilation or on the static gates, not on any model’s self-report. (c) *Final arbiter*: all statements and proofs reported here were checked by the Lean 4 kernel: every proof compiles with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. (d) The human author reviewed and is responsible for all content; the AI system is not an author.

Author contributions

CRediT-style: the human author is responsible for topic adjudication and selection, verification of statement semantics against the frozen files, gate sign-off, and the decision to deposit; the AI4Math pipeline (an LLM-based system) carried out the enumeration scripts, autoformalization, proof search, audit re-runs and drafting of this note. The AI system is not listed as an author.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE-28*, LNCS 12699, pp. 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37

- [2] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP '20)*, pp. 367–381, 2020. <https://doi.org/10.1145/3372885.3373824>
- [3] The OEIS Foundation. Entry A001353: $a(n) = 4a(n - 1) - a(n - 2)$ with $a(0) = 0$ and $a(1) = 1$; the entry’s comment of Joshua Zucker and the Castilleja School Math Club (2003-10-28) on packing a $3 \times (2n - 1)$ rectangle with one extra square attached. *The On-Line Encyclopedia of Integer Sequences*. <https://oeis.org/A001353>
- [4] The OEIS Foundation. Entry A210662: triangle $T(n, k)$, number of monomer-dimer tilings of an $n \times k$ board. *The On-Line Encyclopedia of Integer Sequences*. <https://oeis.org/A210662>